

SPDX-LICENSE-IDENTIFIER: GPL-3.0-OR-LATER

Compiladores PIC16 desde dentro

Libro didáctico ampliado para desarrollar pic16cc por fases

Edición revisada y expandida: frontend, IR, ABI, backend PIC16, helpers, HEX y validación

Documento generado como ampliación pedagógica del material técnico original.

Índice

1. Antes de empezar: qué se ha ampliado y cómo leer este libro
2. 1. Qué es un compilador
3. 2. Las fases de cualquier compilador
4. 3. Frontend y backend
5. 4. PIC16 como caso de estudio
6. 5. El primer paso lógico: compilar algo mínimo
7. 6. Tokenizar: convertir texto en unidades
8. 7. Parsear: convertir tokens en estructura
9. 8. Análisis semántico: nombres, tipos y sentido
10. 9. IR: la representación intermedia
11. 10. Control de flujo: if, while, for y saltos
12. 11. Modelo de memoria: globales, locales, arrays y punteros
13. 12. ABI stack-first y frames
14. 13. Aritmética y helpers runtime
15. 14. Lowering: bajar operaciones abstractas a PIC16
16. 15. Banking y paging: el enemigo silencioso
17. 16. Codificar instrucciones y emitir Intel HEX
18. 17. ISR: interrupciones sin corromper el programa
19. 18. Optimización conservadora
20. 19. Diagnósticos: errores que enseñan
21. 20. Validación: simulador, RAM, SFR y tolerancias
22. 21. Fases 2 a 7: del núcleo mínimo al compilador útil
23. 22. Fases 8 a 18: crecer sin perder el control
24. 23. Fases 19 a 26: validación, 32 bits, fixed-point y configuración
25. 24. Fases 27 a 35: float, paging, catálogo de helpers y compactación
26. 25. Fases 36 a 46: math.h finito, sqrt, min/max y trigonometría
27. 26. Cómo se añade una feature sin romper el compilador
28. 27. Estructura de repositorio y lectura con criterio
29. 28. Diseño de CLI, reports y experiencia de uso
30. 29. Taller guiado: traducir una función completa
31. 30. Glosario ampliado
32. 31. Conclusión: cómo pensar como quien diseña compiladores
33. Apéndice A. Plantilla de tests por fase
34. Apéndice B. Errores frecuentes al construir pic16cc

Antes de empezar: qué se ha ampliado y cómo leer este libro

El documento original funciona como una guía breve por fases: explica que `pic16cc` es un compilador C para PIC16 escrito en Rust, enumera el estado del proyecto y marca una progresión desde el primer programa hasta `math.h` y trigonometría. Lo que faltaba era convertir esa guía en una explicación didáctica: definir cada concepto, mostrar ejemplos intermedios, justificar cada decisión y enseñar cómo un fragmento de C se transforma gradualmente en instrucciones para un microcontrolador PIC16.

Esta edición reestructura el material como libro técnico. Mantiene la idea central del original - aprender compiladores construyendo uno pequeño para PIC16 - pero añade el contexto que normalmente se da por supuesto: tokenización, parsing, AST, análisis semántico, IR, ABI, lowering, banking, paging, codificación de instrucciones, emisión Intel HEX, runtime helpers, validación con simulador y criterios de tamaño.

Objetivo práctico

No se busca memorizar una lista de fases. Se busca que puedas mirar una línea como `return a + 3;` y explicar, capa por capa, qué representación tiene en cada punto del compilador y por qué esa representación existe.

Convenciones usadas

- Los ejemplos de C son deliberadamente pequeños para que se pueda seguir toda la traducción.
- El ensamblador PIC16 mostrado es conceptual cuando se indica. En un compilador real, los detalles exactos se validan contra el datasheet del dispositivo y las reglas del backend.
- Cuando se menciona PIC16 se piensa en la familia mid-range clásica de 14 bits usada como destino didáctico del proyecto.
- El libro usa el enfoque por fases del proyecto `pic16cc`: no intenta soportar todo C desde el día uno.

Mapa rápido del libro

Bloque	Qué aprenderás
Fundamentos	Qué hace un compilador y por qué no es solo cambiar texto por texto.
Frontend	Cómo el compilador lee C: preprocesado, lexer, parser, AST y semántica.
Representación intermedia	Por qué se usa IR y cómo facilita optimizaciones y backend.
Backend PIC16	

Bloque	Qué aprenderás
	Cómo se baja una operación abstracta a bancos, páginas, registros e instrucciones reales.
ABI y runtime	Cómo se pasan argumentos, cómo se devuelve un valor y cuándo hacen falta helpers.
Validación	Cómo demostrar que el compilador genera código correcto y cabe en el target.
Crecimiento por fases	Cómo añadir arrays, punteros, structs, floats, math.h y trigonometría sin romper la base.

1. Qué es un compilador

Un compilador es un programa que transforma un lenguaje fuente, pensado para humanos, en una representación ejecutable por una máquina o por otra herramienta. En este proyecto, la entrada es un subconjunto de C y la salida principal es un fichero Intel HEX que se puede programar en un microcontrolador PIC16.

La definición sencilla - convertir C en ensamblador o HEX - se queda corta. Un compilador serio también debe rechazar programas inválidos, explicar errores de forma útil, decidir dónde vive cada variable, respetar reglas de tipos, generar llamadas, crear saltos, reservar memoria, seleccionar instrucciones y medir si el resultado cabe en el dispositivo.

La traducción no es una sustitución de palabras

Una idea frecuente al empezar es imaginar el compilador como un traductor literal: ve una suma y escribe una instrucción de suma. Eso solo sirve para ejemplos muy pequeños. En realidad, el compilador necesita entender estructura, significado y restricciones del destino.

```
unsigned char add1(unsigned char x) {  
    return x + 1;  
}
```

Esa función parece trivial, pero obliga al compilador a responder varias preguntas: ¿qué es unsigned char?, ¿dónde está x?, ¿qué ocurre si x vale 255?, ¿cómo se devuelve el resultado?, ¿qué registros del PIC16 se pueden tocar?, ¿hay que cambiar de banco?, ¿la suma cabe en una instrucción o necesita ayuda?

Las capas de significado

Capa	Pregunta que responde	Ejemplo con return x + 1
Texto	Qué caracteres aparecen.	r e t u r n, espacio, x, +, 1, ;
Tokens	Qué unidades léxicas reconoce el lenguaje.	RETURN, IDENT(x), PLUS, INTEGER(1), SEMICOLON
AST	Qué estructura sintáctica tiene.	Return(BinaryAdd(Variable(x), Literal(1)))
Tipos	Qué significa cada expresión.	x es unsigned char; 1 se convierte al tipo de la operación.
IR	Qué pasos abstractos ejecuta.	tmp0 = load_arg x; tmp1 = add tmp0, 1; return tmp1
Backend	Cómo se implementa en PIC16.	Cargar W, sumar literal, guardar o devolver en W.
HEX	Qué palabras de programa se graban.	Secuencia de palabras de 14 bits empaquetadas en registros Intel HEX.

Idea clave

Compilar es perder ambigüedad de forma controlada. Cada fase convierte una representación más humana en otra más cercana al hardware, añadiendo información y eliminando opciones imposibles.

Qué hace especial a un compilador para microcontroladores

En un compilador para un PC moderno, muchos problemas quedan escondidos por el sistema operativo, el enlazador, la ABI de la plataforma y una biblioteca estándar grande. En un PIC16 casi todo aparece en primer plano: la RAM es pequeña, la pila hardware es limitada, el banco de memoria importa, la página de programa importa, las instrucciones son simples y las llamadas tienen restricciones.

Por eso PIC16 es un excelente caso de estudio. Obliga a diseñar conscientemente cosas que en otras plataformas se dan por hechas: dónde se guarda un temporal, cómo se hace una multiplicación, cómo se representa un puntero, cómo se preserva el contexto de una interrupción y cómo se valida que el binario final entra en el chip.

2. Las fases de cualquier compilador

Dividir un compilador en fases no es una manía académica: es una estrategia de supervivencia. Cada fase tiene una responsabilidad clara, una entrada clara y una salida clara. Si aparece un error, sabes dónde buscarlo. Si quieres añadir una feature, sabes qué capas toca.

Pipeline general

```
source.c
-> preprocessor
-> lexer
-> parser
-> AST
-> semantic analysis
-> typed AST
-> IR generation
-> IR optimization
-> lowering to PIC16 operations
-> register/RAM/bank/page decisions
-> assembly listing
-> instruction encoding
-> Intel HEX
```

El pipeline no tiene por qué ser exactamente igual en todos los compiladores, pero la separación conceptual sí suele aparecer. Un compilador pequeño puede fusionar fases por simplicidad; uno grande puede dividir las aún más. Lo importante es no mezclar responsabilidades sin necesidad.

Ejemplo completo: $a = b + 3$

```
a = b + 3;
```

La misma línea significa cosas distintas en cada fase. Como texto es solo una secuencia de caracteres. Como tokens es una lista de unidades. Como AST es una asignación cuyo lado derecho es una suma. Como semántica es una operación entre tipos concretos y una escritura en una variable concreta. Como IR es una secuencia de pasos. Como backend es una serie de cargas, sumas y stores en una arquitectura concreta.

Fase	Representación simplificada
Lexer	IDENT(a), ASSIGN, IDENT(b), PLUS, INT(3), SEMICOLON
Parser	Assign(name=a, value=Add(Var(b), Int(3)))
Semántica	a: uchar, b: uchar, 3 representable; asignación válida
IR	t0 = load b; t1 = add8 t0, 3; store a, t1
Backend	bank(a/b); movf b,W; addlw 3; movwf a
HEX	registros Intel HEX con palabras de programa codificadas

Por qué no conviene resolverlo todo de golpe

Si el lexer intenta entender tipos, el parser intenta elegir registros y el backend intenta arreglar errores de sintaxis, el proyecto se vuelve frágil. Un fallo en una capa se manifiesta como un síntoma extraño en otra. En cambio, si cada fase valida su salida, puedes imprimir dumps de tokens, AST, IR y ensamblador para localizar el problema.

Regla de diseño

Una fase debe consumir una representación válida y producir otra representación válida. Si algo no encaja, debe fallar pronto y con un diagnóstico localizado.

Fases mínimas para pic16cc

- Preprocesador: resolver includes simples, macros básicas y condicionales si el proyecto los soporta.
- Lexer: convertir texto en tokens con ubicación de archivo, línea y columna.
- Parser: construir un AST que respete la gramática del subconjunto C.
- Análisis semántico: resolver nombres, scopes, tipos, conversiones y errores.
- IR: representar operaciones de forma más uniforme que el AST.
- Backend PIC16: materializar operaciones usando W, RAM, FSR/INDF, bancos y páginas.
- Codificador: convertir instrucciones en palabras de 14 bits y generar Intel HEX.
- Validación: ejecutar pruebas, simulación y comprobación de tamaño.

3. Frontend y backend

La separación entre frontend y backend es una de las ideas más útiles para mantener un compilador. El frontend entiende el lenguaje fuente. El backend entiende la máquina destino. Entre ambos suele vivir una representación intermedia que evita que cada lado tenga que conocer todos los detalles del otro.

Frontend: entender C

El frontend se encarga de responder preguntas sobre el programa escrito por la persona: qué tokens existen, qué estructura gramatical forman, qué nombres se han declarado, qué tipos tienen, qué conversiones son válidas, qué funciones existen y qué errores debe ver el usuario.

- El preprocesador trabaja antes de la gramática C propiamente dicha.
- El lexer reconoce unidades como identificadores, literales, operadores y palabras clave.
- El parser comprueba si la secuencia de tokens encaja en la gramática.
- El análisis semántico comprueba que el programa tiene sentido más allá de la sintaxis.

Backend: obedecer al PIC16

El backend no debería preguntarse si una variable existe o si una llamada tiene el número correcto de argumentos. Eso ya lo resolvió el frontend. El backend recibe operaciones tipadas y debe decidir cómo ejecutarlas en una máquina concreta.

- Seleccionar instrucciones reales o secuencias de instrucciones.
- Gestionar bancos de RAM y páginas de programa.
- Respetar el ABI de llamadas.
- Emitir listados, mapas, ensamblador o directamente palabras de programa.
- Incluir helpers runtime solo cuando se usan.
- Medir ROM, RAM, stack software y coste de helpers.

La representación intermedia como frontera

Sin IR, el backend tendría que recorrer el AST y conocer demasiadas reglas de C. Con IR, el frontend traduce C a operaciones más simples y explícitas. Por ejemplo, una expresión compleja se puede convertir en una lista de asignaciones temporales.

```
C source:  
  z = (a + b) * (c - 1);
```

```
Possible IR:  
  t0 = load a  
  t1 = load b  
  t2 = add8 t0, t1  
  t3 = load c  
  t4 = sub8 t3, 1
```

```
t5 = mul8 t2, t4  
store z, t5
```

Ese IR todavía no dice si t_0 vive en RAM, en W o en una posición temporal del frame. Esa decisión puede dejarse al backend. Lo importante es que las dependencias y tipos ya están claros.

Separación sana

El frontend debe producir un programa tipado y correcto. El backend debe producir una implementación válida para el target. Si el backend arregla errores de lenguaje, el diseño empieza a oler mal.

4. PIC16 como caso de estudio

PIC16 es incómodo en el buen sentido. No te permite esconder los problemas. Tiene pocos registros de propósito general visibles, un acumulador W central, RAM por bancos, páginas de programa, instrucciones de anchura fija de 14 bits en muchos modelos mid-range y una pila hardware limitada para retornos. Todo esto fuerza a tomar decisiones explícitas.

Qué complica PIC16

Restricción	Consecuencia en el compilador
Acumulador W	Muchas operaciones pasan por W; hay que planificar cargas y stores.
RAM bancaria	Acceder a variables en bancos distintos exige seleccionar banco correctamente.
Páginas de programa	CALL/GOTO pueden requerir preparar bits de página o usar stubs seguros.
Pila hardware limitada	No conviene depender de recursión ni llamadas profundas sin análisis.
Sin multiplicación/división general	Se necesitan helpers o expansiones inline para operaciones complejas.
RAM escasa	Los temporales y frames deben medirse, no suponerse infinitos.
Interrupciones	Una ISR debe preservar contexto para no corromper el programa principal.

Por qué sirve para aprender

En un x86-64 moderno puedes escribir una operación y confiar en que hay registros, pila, ABI, enlazador y runtime robustos. En PIC16 cada elección se nota. Si guardas un temporal de más, consumes RAM. Si olvidas seleccionar banco, escribes en otra dirección. Si llamas a una función sin preparar página, saltas a un lugar incorrecto. Si no salvas STATUS en una interrupción, rompes banderas y resultados.

Ejemplo simple y decisiones ocultas

```
unsigned char add1(unsigned char x) {
    return x + 1;
}

void main(void) {
    TRISB = 0x00;
    PORTB = add1(3);
}
```

Este ejemplo ya fuerza varias decisiones: una llamada a función, paso de argumento, retorno en W, acceso a registros SFR, literal inmediato, y quizá cambio de banco si TRISB y PORTB no están en el mismo banco visible en ese punto.

Principio de humildad

Un compilador para PIC16 debe ser conservador. Es mejor generar una instrucción de más que generar una instrucción elegante que falla por banco, página o contexto.

Modelo mental mínimo de PIC16 para el backend

- W es el acumulador principal: muchas operaciones leen o escriben W.
- Los registros de archivo representan RAM y SFR; no todos están accesibles igual en todo momento.
- STATUS contiene banderas y bits relacionados con banco; tocarlo sin cuidado cambia el significado de instrucciones posteriores.
- PCLATH puede participar en saltos o llamadas a ciertas páginas; ignorarlo genera errores que solo aparecen cuando el programa crece.
- FSR/INDF permiten direccionamiento indirecto y son esenciales para punteros y arrays dinámicos.

5. El primer paso lógico: compilar algo mínimo

Un error clásico al escribir un compilador es empezar por el lenguaje completo. Eso casi siempre acaba en una mezcla de casos especiales. El primer objetivo debe ser minúsculo pero completo: leer un fichero, generar una salida válida y demostrar que todo el recorrido funciona.

Programa mínimo recomendado

```
void main(void) {  
    PORTB = 1;  
}
```

Aunque parezca ridículo, este programa recorre casi todo el esqueleto: entrada, tokens, parser, AST, análisis semántico mínimo, generación de IR, backend, emisión y fichero final. La ventaja es que cada capa puede ser simple al principio.

Qué debe existir antes de añadir features

1. Un lector de archivos con errores claros si el fichero no existe.
2. Un lexer que emita tokens con línea y columna.
3. Un parser que construya AST y falle ante tokens inesperados.
4. Una tabla de símbolos mínima con funciones y variables conocidas.
5. Un generador de IR para asignaciones simples.
6. Un backend capaz de emitir instrucciones para literales y stores.
7. Un codificador o emisor que produzca una salida inspeccionable.
8. Tests que comprueben que el recorrido entero no se rompe.

Por qué el primer programa debe ser completo

Si haces primero un lexer perfecto sin backend, puedes pasar semanas sin saber si las decisiones sirven para generar código. Si haces primero un backend a mano sin frontend, no sabrás si el AST lo alimentará bien. Un vertical slice pequeño - de C a HEX - permite validar arquitectura desde el principio.

Vertical slice

Una rebanada vertical es una feature mínima que atraviesa todas las capas. En un compilador, es más valioso compilar una asignación real de extremo a extremo que tener cien reglas de parser sin backend.

Salida útil desde el día uno

El compilador debe poder mostrar dumps. No son una comodidad; son la forma de aprender y depurar.

```
$ picc --dump-tokens blink.c
$ picc --dump-ast blink.c
$ picc --dump-ir blink.c
$ picc --emit-asm blink.c
$ picc --memory-report blink.c
```

Cuando una prueba falla, estos dumps permiten ver en qué fase se deformó el programa. Si el AST es correcto pero el IR no, el bug está en la bajada a IR. Si el IR es correcto pero el ensamblador escribe el banco equivocado, el bug está en backend.

6. Tokenizar: convertir texto en unidades

Tokenizar, o hacer lexing, es transformar una secuencia de caracteres en una secuencia de tokens. Un token es una unidad con significado léxico: palabra clave, identificador, literal, operador, separador o fin de archivo. El lexer no debería decidir si una suma es válida semánticamente; solo debe reconocer qué piezas hay.

Ejemplo de entrada

```
unsigned char x = 12 + y;
```

Tokens resultantes

Texto	Token	Valor asociado
unsigned	KW_UNSIGNED	-
char	KW_CHAR	-
x	IDENTIFIER	x
=	EQUAL	-
12	INTEGER_LITERAL	12
+	PLUS	-
y	IDENTIFIER	y
;	SEMICOLON	-
<fin>	EOF	-

Qué debe guardar un token

Un token no debería guardar solo su tipo. También necesita ubicación para diagnósticos y, cuando procede, valor. Si un usuario escribe un símbolo inválido, el compilador debe poder señalar dónde ocurrió.

```
struct Token {
    kind: TokenKind,
    lexeme: String,
    line: usize,
    column: usize,
}

enum TokenKind {
    Identifier,
    IntegerLiteral,
    KeywordUnsigned,
    KeywordChar,
```

```
Plus,  
Equal,  
Semicolon,  
EndOfFile,  
}
```

Errores léxicos

Un error léxico ocurre cuando los caracteres no pueden formar ningún token válido. Por ejemplo, un carácter inesperado, un literal mal escrito o un comentario sin cerrar. El lexer debe fallar pronto, antes de que el parser intente interpretar basura.

```
Input:  
    unsigned char x = 12 @ y;  
  
Diagnostic:  
    error: unexpected character '@'  
    --> main.c:1:22
```

Palabras clave frente a identificadores

Una palabra como `return` parece un identificador hasta que se compara contra la tabla de palabras clave. La regla habitual es leer una secuencia alfanumérica y después clasificarla.

```
fn classify_word(text: &str) -> TokenKind {  
    match text {  
        "return" => TokenKind::KeywordReturn,  
        "unsigned" => TokenKind::KeywordUnsigned,  
        "char" => TokenKind::KeywordChar,  
        "void" => TokenKind::KeywordVoid,  
        _ => TokenKind::Identifier,  
    }  
}
```

Qué no debe hacer el lexer

- No debe decidir si una variable ha sido declarada.
- No debe comprobar si una suma entre punteros es válida.
- No debe generar instrucciones PIC16.
- No debe interpretar la precedencia completa de operadores.

Separación importante

El lexer reconoce `IDENT(x) PLUS INT(1)`. El parser decidirá que eso es una expresión de suma. La semántica decidirá si esa suma tiene sentido.

7. Parsear: convertir tokens en estructura

Parsear es transformar una lista lineal de tokens en una estructura jerárquica que respeta la gramática del lenguaje. Esa estructura suele ser un AST, o árbol de sintaxis abstracta. El parser no se limita a leer de izquierda a derecha: debe saber que la multiplicación tiene más prioridad que la suma, que los paréntesis agrupan y que un return pertenece al cuerpo de una función.

Ejemplo de precedencia

```
3 + 4 * 5
```

El resultado correcto no es sumar 3 y 4 primero. La multiplicación tiene mayor precedencia. Por eso el AST debe parecerse a esto:

```
Add
  left: Int(3)
  right: Mul
    left: Int(4)
    right: Int(5)
```

Con paréntesis cambia la estructura:

```
(3 + 4) * 5

Mul
  left: Add
    left: Int(3)
    right: Int(4)
  right: Int(5)
```

Gramática mínima para expresiones

Una forma didáctica de escribir la gramática es separar niveles de precedencia. La siguiente gramática no cubre todo C, pero muestra la idea:

```
expression    -> assignment
assignment    -> identifier "=" assignment | additive
additive      -> multiplicative ("+" | "-") multiplicative)*
multiplicative -> unary ("*" | "/" | "%") unary)*
unary         -> ("!" | "~" | "-") unary | primary
primary       -> integer | identifier | "(" expression ")"
```

Recursive descent

Un parser recursive descent implementa una función por nivel gramatical. Es claro, depurable y adecuado para un compilador educativo. Cada función consume tokens si encajan y devuelve un nodo.

```
fn parse_additive(&mut self) -> Result<Expression, ParseError> {
    let mut expression = self.parse_multiplicative()?;

    while self.matches(TokenKind::Plus) || self.matches(TokenKind::Minus) {
        let operator = self.previous().kind;
        let right_expression = self.parse_multiplicative()?;
        expression = Expression::Binary {
            operator,
            left: Box::new(expression),
            right: Box::new(right_expression),
        };
    }

    Ok(expression)
}
```

Errores sintácticos

Un error sintáctico ocurre cuando los tokens son válidos individualmente pero no forman una construcción válida. Por ejemplo, un paréntesis sin cerrar o un operador sin operando.

```
Input:
    return (a + 3;

Diagnostic:
    error: expected ')' before ';'
    --> main.c:4:17
```

AST concreto frente a AST abstracto

Un árbol de sintaxis concreto conserva casi todo: paréntesis, comas, tokens exactos. Un AST abstracto conserva la estructura semánticamente relevante. Para generar código normalmente no necesitas recordar el token exacto de cada paréntesis; sí necesitas saber qué expresión agrupaba.

Elemento	Se conserva en AST?	Motivo
Literal 3	Sí	Tiene valor semántico.
Operador +	Sí	Define una operación.
Paréntesis que solo agrupan	Normalmente no	Su efecto ya está reflejado en la forma del árbol.
Punto y coma	No como nodo principal	Delimita sentencia, pero no es una operación.
Ubicación fuente	Sí, como metadato	Necesaria para diagnósticos.

La prueba mental

Si puedes imprimir el AST y explicar por qué cada hijo está donde está, el parser empieza a ser mantenible.

8. Análisis semántico: nombres, tipos y sentido

Un programa puede ser sintácticamente correcto y aun así no tener sentido. El análisis semántico comprueba reglas que no se pueden resolver solo con gramática: variables declaradas, tipos compatibles, número de argumentos, retorno válido, uso correcto de punteros y restricciones del subconjunto soportado.

Ejemplo de programa sintácticamente válido pero semánticamente incorrecto

```
unsigned char f(void) {  
    return missing + 1;  
}
```

El parser puede construir el AST: return de una suma entre missing y 1. El problema aparece cuando la semántica busca missing en la tabla de símbolos y no lo encuentra.

```
error: use of undeclared identifier "missing"  
--> main.c:2:12
```

Tabla de símbolos

La tabla de símbolos registra nombres visibles en cada scope. Cada entrada puede guardar nombre, tipo, clase de almacenamiento, ubicación, si es función o variable, y metadatos para backend.

Nombre	Clase	Tipo	Scope	Notas
f	función	unsigned char(void)	global	símbolo invocable
x	parámetro	unsigned char	función f	vive en frame de llamada
PORTB	SFR	volatile unsigned char	global target	dirección fija de dispositivo
tmp	local	unsigned int	bloque	requiere espacio en frame

Scopes

Un scope define dónde es visible un nombre. En C hay scope global, de función y de bloque. Un compilador pequeño puede empezar con global y función, pero pronto necesita bloques para variables locales dentro de if, while o for.

```
unsigned char g;  
  
void main(void) {  
    unsigned char x;
```

```
if (g) {  
    unsigned char y;  
    x = y;  
}  
  
y = 1; // error: y is out of scope  
}
```

Tipos y conversiones

El análisis de tipos decide qué tipo tiene cada expresión y si las conversiones son permitidas. En PIC16 esto no es un detalle cosmético: elegir 8 o 16 bits cambia por completo el código generado.

```
unsigned char a;  
unsigned int b;  
  
b = a + 300;
```

Aquí el literal 300 no cabe en unsigned char, así que la operación debe tratarse al menos como 16 bits. Si el compilador la baja erróneamente a add8, el resultado será incorrecto.

Errores semánticos típicos

- Variable no declarada.
- Declaración duplicada en el mismo scope.
- Asignación incompatible entre puntero y entero no permitida.
- Return con valor en función void o sin valor en función no void.
- Llamada con número incorrecto de argumentos.
- Uso de operador no implementado para un tipo.
- Retornar un puntero a una variable local de stack.

Diagnóstico bueno

No basta con decir "semantic error". El mensaje debe decir qué regla se rompió, dónde, y a ser posible qué esperaba el compilador.

9. IR: la representación intermedia

El AST todavía refleja bastante la forma del C original. El backend necesita algo más regular. La representación intermedia, o IR, traduce estructuras del lenguaje a operaciones simples, explícitas y tipadas. Es el puente entre entender C y generar PIC16.

Por qué no generar ensamblador directamente desde el AST

Se puede para programas muy pequeños, pero pronto aparecen problemas: expresiones anidadas, conversiones, temporales, saltos, short-circuit, llamadas, punteros y optimizaciones. Si cada nodo del AST emite ensamblador directamente, el backend queda lleno de casos especiales.

AST frente a IR

Aspecto	AST	IR
Cercanía al C	Alta	Media o baja
Operaciones	Nodos ricos: if, while, binary, call	Instrucciones simples: load, store, add, branch, call
Tipos	Pueden anotarse tras semántica	Deben estar explícitos
Temporales	Implícitos en expresiones	Explícitos
Control de flujo	Estructurado	Etiquetas y saltos
Uso para optimización	Más difícil	Más uniforme

Ejemplo

```
unsigned char f(unsigned char a, unsigned char b) {
    unsigned char c;
    c = a + b;
    return c;
}
```

```
IR conceptual:
function f(a: u8, b: u8) -> u8
entry:
    t0:u8 = load_arg a
    t1:u8 = load_arg b
    t2:u8 = add8 t0, t1
    store_local c, t2
    t3:u8 = load_local c
    return t3
```

Ese IR ya es mucho más cercano a la máquina, pero todavía no decide posiciones concretas de RAM ni bancos. Esa separación permite optimizar antes de comprometerse con detalles de PIC16.

IR tipada

Cada temporal debe conocer su tipo. No es igual add8 que add16, ni load de puntero que load de byte. En un microcontrolador de 8 bits, el tipo controla cuántos bytes se leen, cuántos se escriben y qué helper puede hacer falta.

```
t0:u16 = zero_extend_u8_to_u16 a
t1:u16 = const_u16 300
t2:u16 = add16 t0, t1
```

IR en forma de tres direcciones

Una forma común es el código de tres direcciones: cada instrucción tiene como mucho una operación principal y produce un temporal. Eso simplifica optimizaciones como propagación de constantes y eliminación de código muerto.

```
t0 = add a, b
t1 = sub c, 1
t2 = mul t0, t1
store z, t2
```

Idea práctica

Un buen IR hace que el backend sea aburrido. Si el backend se vuelve demasiado inteligente, probablemente el IR todavía es demasiado parecido al AST.

10. Control de flujo: if, while, for y saltos

El C fuente usa estructuras de control legibles: if, while, for, do, break y continue. El hardware no entiende esas estructuras; entiende saltos condicionales e incondicionales. El compilador debe convertir control estructurado en etiquetas y ramas.

If simple

```
if (x) {  
    PORTB = 1;  
} else {  
    PORTB = 0;  
}
```

```
IR conceptual:  
    t0 = load x  
    branch_if_zero t0, else_label  
then_label:  
    store PORTB, 1  
    jump end_label  
else_label:  
    store PORTB, 0  
end_label:
```

La condición se convierte en una prueba y un salto. El bloque then salta al final para no caer dentro del else. Esta estructura debe ser correcta incluso cuando los bloques contienen más if o bucles.

While

```
while (x) {  
    x = x - 1;  
}
```

```
IR conceptual:  
loop_test:  
    t0 = load x  
    branch_if_zero t0, loop_end  
loop_body:  
    t1 = sub8 t0, 1  
    store x, t1  
    jump loop_test  
loop_end:
```

Break y continue

break y continue parecen simples, pero requieren que el parser o el generador de IR mantenga un contexto de bucles. break salta al final del bucle actual. continue salta al punto de reevaluación o incremento, según el tipo de bucle.

Sentencia	En while	En for
break	salta a loop_end	salta a loop_end
continue	salta a loop_test	salta a for_increment
return	sale de la función	sale de la función

Condiciones en PIC16

PIC16 usa con frecuencia instrucciones de test y skip, como comprobar un bit o saltar la siguiente instrucción según una bandera. El backend puede convertir una rama abstracta en una secuencia de skip más goto. La dificultad es que las instrucciones de skip afectan solo a la siguiente instrucción, así que el layout importa.

```
Conceptual lowering:
    branch_if_zero t0, target
```

```
Possible PIC16-style sequence:
    movf    t0, W
    btfsc   STATUS, Z
    goto    target
```

Cuidado con el layout

Una rama abstracta puede parecer una instrucción. En PIC16 puede convertirse en dos o tres. Las optimizaciones de salto deben ejecutarse después de conocer tamaños reales.

11. Modelo de memoria: globales, locales, arrays y punteros

El modelo de memoria es una parte central de un compilador para PIC16. En un PC, muchas veces basta con decir "esto va a la pila" o "esto va a la sección data". En PIC16 hay que decidir direcciones, bancos, indirect addressing y coste de cada acceso.

Clases de memoria

Elemento C	Ubicación típica	Reto en PIC16
Global	RAM fija o SFR si es registro del dispositivo	Banco correcto y dirección conocida.
Local automática	Frame en pila software o área temporal	Asignar offset y estimar stack.
Local static	RAM fija con duración global	Inicialización y símbolo persistente.
Array local	Frame o RAM reservada	Acceso por base + índice.
Puntero	Valor de dirección de datos	Materializar FSR/INDF y tamaño del puntero.
Const ROM	Memoria de programa	Lectura especial si el target no lee ROM como RAM.

Arrays

```
unsigned char data[4];  
  
data[2] = 7;
```

Para un índice constante, el backend puede convertir `data[2]` en dirección `base + 2`. Para un índice dinámico, necesita direccionamiento indirecto.

```
data[i] = 7;  
  
Conceptual steps:  
  address = base_address(data) + i  
  FSR = address  
  INDF = 7
```

Punteros

Un puntero no es magia: es un valor que representa una dirección. En un PIC16 pequeño puede bastar un puntero de datos de 8 o 16 bits según el modelo elegido. Lo importante es que el compilador sea coherente: el frontend tipa el puntero, el IR distingue carga indirecta y store indirecto, y el backend sabe materializar FSR/INDF.

```
unsigned char read_ptr(unsigned char *p) {  
    return *p;  
}
```

IR conceptual:

```
t0:ptr_u8 = load_arg p  
t1:u8 = load_indirect t0  
return t1
```

Puntero a local: un diagnóstico necesario

```
unsigned char *bad(void) {  
    unsigned char x;  
    return &x;  
}
```

Este programa debería rechazarse o advertirse como error serio en el subconjunto del proyecto. `x` vive en el frame de la función. Al retornar, ese frame deja de ser válido. El puntero apuntaría a memoria reutilizable.

Regla práctica

En un compilador pequeño, soporta primero punteros a objetos con vida clara: globales, arrays y argumentos. Añade casos más complejos solo cuando el análisis de vida y el ABI puedan sostenerlos.

12. ABI stack-first y frames

ABI significa Application Binary Interface: el contrato binario entre quien llama y la función llamada. Define cómo se pasan argumentos, dónde se devuelve el resultado, qué registros o memoria se preservan y cómo se organiza un frame de llamada. Sin ABI, cada llamada se vuelve un caso especial.

Por qué hace falta una ABI aunque el compilador sea pequeño

Si solo compilas main sin funciones, puedes emitir código lineal. En cuanto aparece una llamada, necesitas un acuerdo. La función llamada debe saber dónde están sus argumentos; quien llama debe saber dónde recoger el retorno; ambos deben coincidir sobre quién limpia temporales y cómo se restaura el contexto.

Stack-first ABI

En el enfoque stack-first, los argumentos se empujan a una pila software. La función llamada crea un frame con offsets estables para argumentos, locales y temporales. Esto no es lo más pequeño posible, pero es robusto y didáctico: convierte las llamadas en un mecanismo uniforme.

```
Caller concept:
  push arg0
  push arg1
  call function
  pop args
  use return value

Callee concept:
  save caller frame pointer
  set frame pointer
  allocate locals and temporaries
  ... body ...
  restore frame pointer
  return
```

Layout conceptual de frame

```
higher addresses
+-----+
| caller temporaries |
+-----+
| argument n         |
| ...                |
| argument 0         |
+-----+ <- frame pointer base for args
| saved caller FP    |
+-----+
| local variables    |
| compiler temporaries |
```

+-----+
lower / next stack area

Retornos

Tipo de retorno	Convención posible
void	No se devuelve valor.
8 bits	W contiene el resultado.
16 bits	W contiene byte bajo y un registro runtime contiene byte alto, o se usa una convención equivalente.
Puntero de datos	Se devuelve como valor de 16 bits si el modelo lo requiere.
Struct grande	Puede requerir puntero oculto de retorno; mejor dejarlo para fases avanzadas.

Coste del ABI

Una ABI uniforme tiene coste: más stores, más RAM temporal y más código de prólogo/epílogo. Pero permite implementar funciones, recursión limitada si se decide soportarla, llamadas con varios argumentos y un backend menos caótico. En un compilador didáctico, esa claridad compensa.

Decisión de diseño

Un ABI no se improvisa tarde. Si al principio no defines cómo se llaman las funciones, acabarás añadiendo parches en cada helper, llamada y retorno.

13. Aritmética y helpers runtime

PIC16 no ofrece de forma cómoda todas las operaciones que C permite escribir. Sumar 8 bits puede ser directo. Multiplicar, dividir, calcular módulo, desplazar 16 bits o manejar signed puede requerir secuencias largas o helpers runtime.

Qué es un helper runtime

Un helper runtime es una rutina auxiliar que el compilador emite o enlaza cuando necesita implementar una operación que no cabe bien como instrucciones inline. Por ejemplo, una multiplicación de 16 bits, una división unsigned, una comparación de 32 bits o una conversión float.

Operación C	Posible implementación
a + b en u8	Inline con W y addwf/addlw.
a * b en u8	Helper shift-and-add o inline para constantes pequeñas.
a / b en u16	Helper de división binaria.
a << 3	Secuencia de shifts inline si el conteo es constante.
a << n	Helper o bucle si el conteo es dinámico.
float sinf(x)	Helper math + tablas + perfil de precisión.

Ejemplo: multiplicación shift-and-add

Una multiplicación unsigned puede implementarse sumando el multiplicando cuando el bit correspondiente del multiplicador está activo y desplazando en cada paso.

```
uint8_t multiply_u8(uint8_t left, uint8_t right) {
    uint8_t result = 0;

    while (right != 0) {
        if ((right & 1) != 0) {
            result = result + left;
        }

        left = left << 1;
        right = right >> 1;
    }

    return result;
}
```

El helper real en ensamblador debe respetar el ABI: dónde recibe operandos, dónde deja resultado, qué temporales usa y qué registros preserva. Además debe marcarse como dependencia para que el emisor lo incluya solo si se usa.

Selección entre inline y helper

Criterio	Inline	Helper
Tamaño de código si se usa una vez	Puede ser menor	Tiene coste de llamada.
Tamaño si se usa muchas veces	Crece repetido	Se comparte.
Velocidad	Puede ser más rápido para constantes	Más estable pero con overhead.
Complejidad de backend	Más casos especiales	Centraliza la lógica.
Depuración	Difícil si se expande mucho	Rutina inspeccionable.

División por cero

Si el divisor es una constante cero, el compilador debe diagnosticarlo. Si el divisor es dinámico, el proyecto puede definir una política simple, por ejemplo devolver 0. Lo importante es documentarlo y testearlo; no dejarlo como comportamiento accidental.

```
unsigned char a = 10 / 0; // diagnostic at compile time
unsigned char b = 10 / x; // runtime policy if x can be zero
```

Runtime pequeño, reglas claras

Cuanto menos helpers tenga el runtime, más importante es que cada uno tenga contrato escrito: argumentos, retorno, temporales, flags, bancos, páginas y coste estimado.

14. Lowering: bajar operaciones abstractas a PIC16

Lowering es el proceso de convertir una operación de alto nivel o de IR en operaciones más cercanas al hardware. No es todavía necesariamente codificación binaria; es decidir qué secuencia de instrucciones o micro-operaciones implementa una intención.

Ejemplo de lowering de suma

```
IR:
    t2:u8 = add8 t0, 3
```

```
Lowering conceptual:
    load t0 into W
    add literal 3 to W
    store W into t2
```

```
PIC16-style conceptual assembly:
    movf    t0, W
    addlw   0x03
    movwf   t2
```

Lowering de asignación a SFR

```
PORTB = 0x55;
```

```
Conceptual:
    select bank containing PORTB if needed
    movlw   0x55
    movwf   PORTB
```

La línea C no muestra bancos, pero el backend debe conocerlos. El lowering no puede ignorar que PORTB y TRISB pueden vivir en bancos diferentes según el dispositivo.

Lowering con punteros

```
*p = value;
```

```
Conceptual:
    address = load p
    FSR = address
    W = load value
    INDF = W
```

Aquí aparece una diferencia importante: una asignación directa a variable y una asignación indirecta usan mecanismos distintos. Por eso el IR debe distinguir `store_local`, `store_global` y `store_indirect`.

No todo lowering es local

Algunas decisiones dependen de información global: si una llamada cruza página, si un helper ya se emitió, si una variable cae en un banco concreto o si un salto corto puede reemplazar a uno largo. Por eso el backend suele tener varias subfases: selección, layout, corrección de páginas/bancos y emisión final.

Frontera mental

Lowering responde "cómo se implementa esta operación". Encoding responde "qué bits exactos representa esta instrucción".

15. Banking y paging: el enemigo silencioso

Banking y paging son dos fuentes clásicas de errores en PIC16. Un programa pequeño puede funcionar durante semanas y romperse de golpe al crecer, porque una variable cae en otro banco o una función cruza una página de programa. El compilador debe tratar estos problemas como parte central del backend.

Banking de RAM

La RAM de muchos PIC16 se organiza en bancos. Una instrucción que accede a una dirección de archivo puede interpretar esa dirección en el banco actualmente seleccionado. Por tanto, antes de acceder a un símbolo, el backend debe garantizar que el banco activo es correcto o que usa un mecanismo de acceso seguro.

```
Conceptual bank-safe store:
    ensure_bank(PORTB)
    movlw    0x01
    movwf   PORTB
```

Optimizar sin romper

Emitir `ensure_bank` antes de cada acceso es seguro pero puede inflar el código. Optimizar banking consiste en eliminar cambios redundantes sin asumir demasiado. Si el backend sabe que el banco activo ya es el de `PORTB` y ninguna instrucción intermedia lo cambió, puede omitir la selección.

```
Before:
    banksel PORTB
    movwf   PORTB
    banksel PORTB
    clrf    PORTB

After:
    banksel PORTB
    movwf   PORTB
    clrf    PORTB
```

Paging de programa

Las llamadas y saltos largos pueden depender de bits de página. Si el compilador emite una llamada sin preparar correctamente la página, el programa puede saltar a una dirección incorrecta. Esto suele aparecer cuando el binario ya no cabe en una página inicial.

Stubs page-safe

Una estrategia conservadora es usar stubs o secuencias page-safe para llamadas a runtime helpers e ISR. Es más código, pero evita que el layout final cambie el significado de una llamada.

Problema	Síntoma	Solución de compilador
Banco incorrecto	Se escribe otro registro o variable	banksel/estado de banco verificado
Página incorrecta	call/goto a dirección equivocada	setpage, stubs o relocación validada
Optimización agresiva	Falla solo en programas grandes	invariantes y tests de layout
ISR cambia PCLATH/STATUS	El main falla tras interrupciones	save/restore completo

Regla de oro

Banking y paging no son optimizaciones: son corrección. Primero hazlo seguro; después elimina redundancias con pruebas.

16. Codificar instrucciones y emitir Intel HEX

Codificar es convertir una instrucción ya seleccionada en la palabra binaria que entiende el microcontrolador. En muchos PIC16 mid-range clásicos, cada instrucción de programa ocupa una palabra de 14 bits. El fichero Intel HEX no guarda texto de ensamblador: guarda bytes que representan esas palabras de programa en un formato de registros con checksum.

Tres niveles distintos

Nivel	Ejemplo	Quién lo produce
IR	store PORTB, 1	Generador de IR
Ensamblador conceptual	movlw 0x01; movwf PORTB	Backend / pretty printer
Palabra codificada	opcode + operandos en 14 bits	Codificador
Intel HEX	:10....CC	Emisor de fichero

Ejemplo conceptual de codificación

La codificación real depende de la instrucción y del core concreto. La idea general es que una plantilla de bits reserva campos para opcode y operandos.

```
Instruction model:
  MovLiteralToW { literal: 0x55 }

Conceptual encoding pattern:
  opcode_bits | literal_bits

Result:
  one 14-bit program word
```

Por qué no basta con imprimir ensamblador

Un compilador puede emitir ensamblador y delegar codificación a un ensamblador externo. pic16cc, según el objetivo descrito, busca una salida directa como HEX. Eso obliga a implementar codificación, layout, direcciones, checksum y registros de fin.

Formato Intel HEX

Intel HEX representa datos binarios como líneas ASCII. Cada línea contiene longitud, dirección, tipo de registro, datos y checksum. Para PIC16 hay que tener cuidado con cómo se empaquetan palabras de programa de 14 bits en bytes del HEX según la convención usada por la herramienta de programación.

```
:LLAAAATT[DD...]CC
```

LL	number of data bytes
AAAA	address field
TT	record type
DD	data bytes
CC	checksum

Checksum

El checksum permite detectar líneas corruptas. Se calcula para que la suma de longitud, dirección, tipo, datos y checksum sea cero módulo 256. Si el checksum está mal, el programador debe rechazar la línea.

```
def compute_intel_hex_checksum(byte_values):  
    total = sum(byte_values) & 0xFF  
    checksum = ((~total + 1) & 0xFF)  
    return checksum
```

Verificación esencial

Después de emitir HEX, el compilador debe poder volver a leerlo o compararlo con una representación esperada. Un error de endianness o dirección puede dejar el listado ASM perfecto y el HEX inútil.

17. ISR: interrupciones sin corromper el programa

Una interrupción rompe el flujo normal: el hardware salta a una rutina especial cuando ocurre un evento. En PIC16, la ISR suele vivir en un vector fijo, por ejemplo 0x0004 en la familia clásica. El compilador debe generar una entrada y salida que preserve el contexto que el programa principal espera conservar.

Por qué una ISR no es una función normal

Una función normal se llama en un punto conocido y respeta el ABI normal. Una ISR puede ocurrir entre dos instrucciones del main. Si modifica W, STATUS, PCLATH, FSR o temporales runtime sin restaurarlos, el main continúa con estado corrupto.

Prólogo y epílogo conceptual

```
ISR entry:
    save W
    save STATUS
    save PCLATH
    save FSR
    save runtime scratch registers
    ... user ISR body ...
    restore runtime scratch registers
    restore FSR
    restore PCLATH
    restore STATUS
    restore W
    retfie
```

Restricciones del cuerpo ISR

Una estrategia didáctica y segura es limitar lo que se permite dentro de una ISR al principio: operaciones inline simples, acceso a SFR, comparaciones, bit operations y stores directos. Llamadas normales y helpers complejos pueden prohibirse hasta que exista un protocolo robusto para reentrada y preservación de contexto.

Permitido al principio	Motivo
Leer/escribir SFR	Caso de uso principal de interrupciones.
Operaciones 8-bit simples	Se pueden emitir inline.
Comparaciones y ramas	Necesarias para flags y despacho.
Uso limitado de locales	Si el frame/temporales están controlados.

Restringido al principio	Riesgo
Llamadas normales	Pueden romper pila, PCLATH o ABI si no se preserva todo.

Restringido al principio	Riesgo
Helpers runtime	Pueden no ser reentrantes.
Float/math compleja	Demasiados temporales y estado compartido.
Recursión	Incompatible con análisis simple de stack.

Ejemplo

```
void __interrupt isr(void) {
    if (INTCON & 0x02) {
        PORTB = PORTB ^ 0x01;
        INTCON = INTCON & (unsigned char)~0x02;
    }
}
```

Objetivo de seguridad

Una ISR correcta no solo hace su trabajo. También deja el mundo exactamente como estaba para el código interrumpido, excepto por los cambios explícitos que el programa quería.

18. Optimización conservadora

Optimizar no significa arriesgar la corrección para ahorrar una instrucción. En un compilador para microcontroladores, la optimización debe ser conservadora, medible y fácil de desactivar. Primero se genera código correcto; después se eliminan redundancias demostrables.

Optimizaciones útiles en pic16cc

Optimización	Ejemplo	Beneficio
Constant folding	<code>2 + 3 -> 5</code>	Menos instrucciones y helpers.
Constant propagation	<code>x = 4; y = x + 1 -> y = 5</code> si x no cambia	Simplifica IR.
Dead code elimination	Eliminar temporales no usados	Menos RAM y ROM.
Branch simplification	<code>if (0)</code> elimina rama	Código más pequeño.
Peephole	<code>movwf a; movf a,W</code> redundante en ciertos contextos	Menos instrucciones locales.
Bank/page cleanup	Eliminar <code>banksel/setpage</code> repetidos seguros	Ahorro importante en PIC16.

Constant folding

```
Before IR:
    t0 = const 2
    t1 = const 3
    t2 = add8 t0, t1
    store x, t2

After IR:
    store x, const 5
```

El folding debe respetar el tipo y el comportamiento definido por el proyecto. Un overflow en unsigned char puede envolver a 8 bits si esa es la semántica definida. Un overflow signed puede ser rechazado, definido de forma propia o tratado con cuidado según el subconjunto.

Eliminación de código muerto

```
Before:
    t0 = load a
    t1 = add8 t0, 1
    t2 = add8 t0, 2
    return t1

After:
    t0 = load a
```

```
t1 = add8 t0, 1
return t1
```

Peephole en backend

Peephole mira ventanas pequeñas de instrucciones y reemplaza patrones seguros. Es tentador meter mucha inteligencia aquí, pero debe usarse con disciplina. Si una regla depende de flags, bancos o W, debe documentar qué preserva.

```
Pattern:
    movlw 0x00
    movwf x
```

```
Replacement:
    clrf x
```

Valid only if the flag side effects are acceptable in that context.

Trampa común

En PIC16, una instrucción más corta puede cambiar banderas. Si una optimización altera Z, C, DC o banco activo y luego el programa depende de ello, has introducido un bug.

Informes de optimización

Un compilador didáctico debería poder explicar qué optimizó. Un `--opt-report` ayuda a ver si una fase hizo lo que esperabas y si una regresión aumentó tamaño.

```
Optimization report:
  constants folded: 12
  dead temporaries removed: 7
  redundant banksel removed: 5
  helpers pruned: __rt_mul_u8 unused
```

19. Diagnósticos: errores que enseñan

Un compilador no solo traduce programas correctos. También guía al usuario cuando el programa está mal. Los diagnósticos son parte de la calidad del compilador, no un accesorio. En un proyecto educativo, además, son una herramienta para comprobar que cada fase entiende su responsabilidad.

Anatomía de un buen error

Parte	Función
Severidad	error, warning, note.
Mensaje principal	Qué regla se rompió.
Ubicación	Archivo, línea y columna.
Fragmento fuente	Línea afectada con marcador.
Notas	Información secundaria: declaración previa, tipo esperado, sugerencia.

```
error: cannot assign unsigned int to unsigned char without narrowing
--> main.c:6:9
6 |      x = 300;
  |      ^^^ value does not fit in unsigned char
note: x was declared here as unsigned char
--> main.c:3:19
```

Errores por fase

Fase	Ejemplo de error
Lexer	Carácter inesperado o literal mal cerrado.
Parser	Paréntesis o punto y coma esperado.
Semántica	Nombre no declarado, tipo incompatible, llamada inválida.
IR	Operación no representable por el subconjunto.
Backend	Feature válida en C pero no implementada para target.
Layout	Programa no cabe en ROM o RAM.
HEX	Error interno de dirección, checksum o rango.

Warnings útiles

Los warnings deben ser accionables. Un warning que el usuario no puede corregir se convierte en ruido. En un compilador para PIC16, son especialmente útiles los avisos sobre narrowing, código inalcanzable, uso de variables no inicializadas si se soporta análisis, y features que generan helpers grandes.

```
warning: division requires runtime helper __rt_div_u16
--> control.c:14:16
14 |         duty = a / b;
    |                   ^ this may increase ROM by 96 words
```

Modo -Werror

Soportar -Werror obliga a que los warnings sean serios. Si el compilador emite warnings espurios, los usuarios no podrán activar este modo en CI. Por eso conviene añadir pruebas específicas de diagnóstico: mensaje, ubicación y ausencia de warning cuando el código es correcto.

Criterio de calidad

Un error bueno reduce el tiempo de depuración. Un error malo obliga al usuario a depurar el compilador mentalmente.

20. Validación: simulador, RAM, SFR y tolerancias

Un compilador no está terminado cuando genera un HEX. Está terminado cuando puedes demostrar que ese HEX ejecuta el comportamiento esperado en el target o en una simulación fiel. La validación debe acompañar cada fase del proyecto.

Niveles de prueba

Nivel	Qué comprueba
Unit tests de lexer	Texto -> tokens correctos.
Parser tests	Tokens -> AST esperado o error esperado.
Semantic tests	Tipos, scopes y diagnósticos.
IR snapshot tests	AST tipado -> IR estable.
Backend tests	IR -> ASM esperado o propiedades.
Encoding tests	Instrucción -> palabra de 14 bits esperada.
HEX tests	Registros, direcciones y checksum.
Simulator tests	Programa ejecutado -> RAM/SFR con valores esperados.
Size tests	ROM/RAM/helpers dentro de límites.

Harness de simulación

Un harness de simulación compila un programa, lo ejecuta en un simulador como pic16-sim y lee memoria o registros al final. Esto permite validar semántica real, no solo texto generado.

Test idea:

1. Generate C test case.
2. Compile with picc to HEX.
3. Run HEX in simulator until stop condition.
4. Read RAM address or SFR.
5. Compare with expected value.

Ejemplo

```
unsigned char result;

void main(void) {
    result = (3 * 4) + 5;
}
```

```
Expected after simulation:  
result == 17
```

Pruebas de math con tolerancias

Cuando aparecen float, sqrt, min/max y trigonometría, la validación deja de ser igualdad exacta en muchos casos. Hay que definir tolerancias por perfil: compacto, balanced o precise. Una función sinf compacta puede aceptar más error si ahorra ROM; una balanced debe ser más precisa y no peor en rejillas definidas.

Perfil	Objetivo	Validación típica
compact	ROM baja	Tolerancia mayor y rango limitado documentado.
balanced	Compromiso	Menor tolerancia y más puntos de prueba.
precise	Máxima precisión razonable	Puede delegarse o aplazarse si el coste es alto.

Medir siempre

En firmware, correcto no basta: también debe caber. Cada test importante debería poder registrar ROM, RAM, stack estimado y helpers incluidos.

21. Fases 2 a 7: del núcleo mínimo al compilador útil

Las fases iniciales convierten el proyecto de una demostración mínima en un compilador pequeño pero real. La progresión correcta no consiste en añadir sintaxis al azar, sino en reforzar el núcleo: tipos de 16 bits, comparaciones, memoria, ABI, helpers, ISR y optimización conservadora.

Fase 2: int, unsigned int y comparaciones

Salir del mundo de 8 bits es decisivo. PIC16 es una máquina de 8 bits, pero C necesita comparar y operar valores de 16 bits. Esto obliga a representar valores multi-byte, definir orden de bytes, generar comparaciones por byte y manejar carry/borrow.

```
unsigned int a;
unsigned int b;

if (a < b) {
    PORTB = 1;
}
```

Una comparación de 16 bits no es una única instrucción. El backend puede comparar byte alto primero y, si son iguales, byte bajo. Para signed, la regla cambia.

Fase 3: arrays, punteros y modelo de memoria

Aquí aparece memoria como concepto central. Arrays introducen base + offset; punteros introducen direcciones como valores; FSR/INDF permiten acceso indirecto. Esta fase suele revelar si el IR y el ABI estaban bien diseñados.

Fase 4: stack-first ABI y frames

Con tres o más argumentos, llamadas anidadas, locales y temporales por función, hace falta un contrato. El stack-first ABI simplifica razonamiento: cada función tiene un frame y offsets estables. El coste se puede medir y optimizar después.

Fase 5: helpers aritméticos

Multiplicación, división, módulo y shifts dinámicos convierten el compilador en una herramienta más usable. Pero cada helper debe tener contrato ABI, tests, coste y poda: no se emite si no se usa.

Fase 6: ISR

La interrupción introduce una ruta especial de ejecución. El compilador debe generar vector, prólogo, epílogo y restricciones semánticas para evitar corrupción de estado.

Fase 7: optimización conservadora

Una vez el compilador genera código correcto, se empiezan a eliminar redundancias: folding, propagación, DCE, peephole y limpieza de banking/paging. La regla es clara: ninguna optimización debe cambiar semántica ni flags relevantes.

Lección de estas fases

Antes de crecer el lenguaje, hay que estabilizar las bases: tipos, memoria, llamadas, runtime, interrupciones y pruebas.

22. Fases 8 a 18: crecer sin perder el control

Después del núcleo mínimo, el proyecto empieza a soportar partes más expresivas de C. El reto es no perder el modelo interno. Cada feature debe entrar porque el IR, la semántica y el backend ya tienen un lugar natural para ella.

typedef, enum y struct

typedef no crea un tipo nuevo en el sentido físico; crea un alias de nombre. enum introduce constantes nombradas. struct introduce layout: offsets de campos, alineación si se define, tamaño total y acceso por punto o flecha.

```
typedef unsigned char byte;

struct Pair {
    byte a;
    byte b;
};

struct Pair p;
p.b = 7;
```

El acceso p.b se convierte en dirección base de p + offset del campo b. En este ejemplo, si a ocupa 1 byte, b puede estar en offset 1. En structs más complejos hay que calcular layout cuidadosamente.

switch

switch parece sintaxis cómoda, pero puede bajarse a cadenas de comparaciones, tablas de salto o estrategias híbridas. En PIC16, una cadena de comparaciones suele ser el primer paso razonable. Fallthrough y default deben modelarse explícitamente.

```
switch (mode) {
case 0:
    PORTB = 1;
    break;
case 1:
    PORTB = 2;
    break;
default:
    PORTB = 0;
    break;
}
```

String literals e inicialización estática

Los literales de cadena abren una pregunta importante: ¿viven en RAM o ROM? En microcontroladores, copiar todo a RAM puede ser imposible. Por eso una fase posterior introduce lectura ROM y tablas constantes.

Preincremento, postincremento y asignaciones compuestas

Operadores como `++`, `+=` y `-=` no son difíciles por sintaxis, sino por semántica: preincremento y postincremento devuelven valores distintos. El IR debe capturar cuándo se lee el valor antiguo y cuándo se escribe el nuevo.

```
x = i++;
```

Conceptual IR:

```
old = load i
new = add old, 1
store i, new
store x, old
```

Punteros compatibles

Comparar, asignar o restar punteros exige reglas. Un compilador pequeño puede soportar solo comparaciones de igualdad entre punteros compatibles y rechazar aritmética compleja hasta tener un modelo sólido.

Function pointers

Los punteros a función son especialmente delicados en PIC16 porque el espacio de código y el de datos no son equivalentes y las llamadas indirectas pueden no estar soportadas de forma natural. Una implementación segura puede usar trampolines o tablas de despacho con IDs en vez de punteros crudos.

Criterio de madurez

Una feature nueva debe añadir reglas a la semántica, representación al IR, lowering al backend y tests. Si solo se añade al parser, todavía no está implementada.

23. Fases 19 a 26: validación, 32 bits, fixed-point y configuración

Estas fases convierten el compilador en una herramienta de firmware más seria. Ya no basta con soportar sintaxis: hay que validar ejecución, manejar tipos más grandes, introducir números no enteros de forma controlada, estimar recursos y configurar el dispositivo correctamente.

Simulación y ejecución validada

La simulación permite comprobar que el HEX modifica RAM y SFR como se espera. Es especialmente importante para ramas, llamadas, helpers e ISR, donde un listado textual correcto puede ocultar errores de estado.

long y unsigned long

Los valores de 32 bits son costosos en una arquitectura de 8 bits. Cada operación puede requerir cuatro bytes, propagación de carry, comparaciones por partes y helpers específicos. La semántica debe promocionar tipos correctamente y el backend debe medir coste.

```
unsigned long acc;  
unsigned int value;  
  
acc = acc + value;
```

Esta suma implica extender value a 32 bits y sumar byte a byte con carry. No es un detalle menor: afecta rendimiento, ROM y RAM temporal.

Fixed-point

Fixed-point representa números con una escala fija, por ejemplo Q8.8: 8 bits de parte entera y 8 bits de fracción. Es más barato que float y muy útil en firmware para sensores, PWM, control y animaciones.

```
Conceptual Q8.8:  
raw 256 -> 1.0  
raw 384 -> 1.5  
raw 128 -> 0.5  
  
Addition:  
raw_result = raw_a + raw_b  
  
Multiplication:  
raw_result = (raw_a * raw_b) >> 8
```

Recursos y límites del target

Un compilador embebido debe informar si el programa cabe. ROM, RAM, stack software, profundidad de llamadas y helpers no son métricas decorativas. Son condiciones de despliegue.

```
Memory report example:
  program ROM: 812 / 2048 words
  globals RAM: 22 bytes
  max frame: 9 bytes
  estimated software stack: 18 bytes
  helpers: __rt_mul_u8, __rt_div_u16
```

Configuración y HEX final

La configuración del PIC - oscillator, watchdog, low-voltage programming, brown-out, etc. - puede estar representada como pragmas, opciones CLI o símbolos especiales. El HEX final debe incluir config words válidas para el dispositivo seleccionado.

Firmware real

Un compilador para microcontroladores debe producir binarios correctos y configurados. Un programa perfecto con config bits incorrectos puede no arrancar.

24. Fases 27 a 35: float, paging, catálogo de helpers y compactación

Float en PIC16 es una decisión pesada. Permite escribir código más cómodo, pero exige runtime, comparaciones, conversiones, constantes, operaciones y validación. Estas fases tienen sentido solo cuando el compilador ya controla tamaño, llamadas, helpers y layout.

Float como valor de 32 bits

Un float típico ocupa 32 bits. En una máquina de 8 bits eso implica cuatro bytes por valor, helpers para casi cada operación y cuidado con temporales. El compilador debe decidir si soporta literales, variables, comparaciones y conversiones antes de prometer compatibilidad amplia.

Constantes float

Un literal como 1.5f debe convertirse a su representación binaria en compilación. Después, el backend debe materializar esos bytes en RAM o ROM según el uso.

```
float x = 1.5f;

Conceptual lowering:
  bytes = encode_float32(1.5)
  store x[0], bytes[0]
  store x[1], bytes[1]
  store x[2], bytes[2]
  store x[3], bytes[3]
```

Comparaciones float

Comparar float no es comparar cuatro bytes como entero sin más. Hay que respetar signo, exponente, mantisa y políticas sobre NaN/Inf si el proyecto las soporta. En un perfil finito y simplificado, se pueden restringir casos y documentarlo.

Seguridad de página y layout

Al crecer runtime helpers de float, aumenta el riesgo de cruzar páginas de programa. Por eso las fases de paging y layout son compañeras naturales de float: cuanto más grande el binario, más importante el control de llamadas y saltos.

Catálogo de helpers

Un catálogo central de helpers evita duplicación. Cada helper debe registrar nombre, perfil, dependencias, tamaño, tipos de entrada, salida y restricciones de target. Así el compilador puede podar helpers no usados y seleccionar variantes compactas o precisas.

Campo	Ejemplo
Nombre	<code>__rt_f32_add</code>
Entradas	dos f32 según ABI runtime
Salida	f32 en buffer o registros definidos
Dependencias	<code>__rt_u32_shift</code> , <code>__rt_norm</code>
Perfil	compact/balanced/precise
Tamaño estimado	palabras ROM aproximadas
Restricciones	finite-only, no erno, no fenv

Compactación

Cuando se tienen helpers grandes, la compactación importa. Puede significar compartir núcleos comunes, eliminar tablas no usadas, seleccionar variantes por perfil o reutilizar rutinas enteras ya existentes.

Lección

Float no es solo un tipo. Es una pequeña biblioteca matemática, una política de precisión y un problema de tamaño.

25. Fases 36 a 46: math.h finito, sqrt, min/max y trigonometría

Las fases finales descritas en el documento original muestran una evolución muy interesante: convertir operaciones matemáticas flotantes en una biblioteca pequeña, medible y validada. En un microcontrolador no se puede importar math.h como si hubiera un sistema operativo grande detrás. Hay que elegir un subconjunto.

math.h mínimo

Un subconjunto razonable puede incluir fabsf, truncf, floorf, ceilf, roundf, sqrtf, fminf, fmaxf, sinf y cosf. Cada función debe tener contrato: dominio, rango, precisión esperada, casos no soportados y coste.

Función	Decisión didáctica
fabsf	Limpiar bit de signo; helper pequeño.
truncf/floorf/ceilf/roundf	Conversión controlada de fracción según modo.
sqrtf	Helper compacto con perfiles de precisión.
fminf/fmaxf	Reutilizar comparación float bien probada.
sinf/cosf	Tablas, simetrías y rango limitado documentado.

Perfiles de precisión

Los perfiles permiten que el usuario elija entre tamaño y precisión. compact puede usar tablas pequeñas y tolerancia mayor. balanced puede usar más puntos, mejor interpolación o correcciones. precise puede aplazarse si el coste no encaja en el proyecto.

sqrtf

sqrtf es una buena prueba porque combina algoritmo numérico, precisión y coste. El compilador debe validar contra valores esperados, no solo comprobar que genera código. También debe documentar qué ocurre con negativos, NaN o infinito si el runtime es finite-only.

fminf y fmaxf

Estas funciones enseñan reutilización: si ya existe una comparación float correcta, fminf y fmaxf deberían apoyarse en ella. Duplicar lógica aumenta tamaño y riesgo.

sinf y cosf con núcleo compartido

sinf y cosf están relacionados. Compartir un núcleo reduce duplicación: wrappers pequeños preparan modo o desfase, y un core común calcula el resultado. Esta idea es muy valiosa en

microcontroladores, donde dos helpers casi iguales pueden duplicar cientos de palabras ROM.

```
Conceptual design:
__rt_f32_sin(x):
    push x
    push mode_sin
    call __rt_f32_sincos_core

__rt_f32_cos(x):
    push x
    push mode_cos
    call __rt_f32_sincos_core
```

Reducción de rango moderada

La trigonometría exige reducir ángulos. Una reducción completa y precisa puede ser cara. Una estrategia didáctica es validar un rango principal, por ejemplo $[-2\pi, +2\pi]$, y añadir alias deterministas para múltiplos concretos como $\pm 3\pi$ o $\pm 4\pi$. Lo importante es declarar el contrato y medir error.

Conclusión numérica

En microcontroladores, una función matemática no es correcta en abstracto. Es correcta para un contrato de dominio, tolerancia, perfil y coste.

26. Cómo se añade una feature sin romper el compilador

La forma profesional de ampliar un compilador es tratar cada feature como un cambio transversal controlado. No basta con añadir una regla al parser. Hay que pensar en todo el recorrido: texto, tokens, AST, semántica, IR, backend, runtime, diagnósticos, tests y documentación.

Checklist de feature

Capa	Pregunta
Lexer	¿Hace falta un token nuevo?
Parser	¿Qué gramática cambia? ¿Hay precedencia nueva?
AST	¿Hace falta un nodo nuevo o basta con uno existente?
Semántica	¿Qué tipos acepta? ¿Qué errores debe emitir?
IR	¿Qué operación representa la feature?
Optimización	¿Hay folding o simplificación segura?
Backend	¿Qué secuencia PIC16 la implementa?
Runtime	¿Necesita helper? ¿Se poda si no se usa?
Validación	¿Hay tests de éxito, error y simulación?
Documentación	¿Queda claro el límite soportado?

Ejemplo: añadir operador módulo para unsigned char

1. El lexer ya reconoce %. Si no, se añade token Percent.
2. El parser lo coloca en el nivel multiplicative junto a * y /.
3. El AST usa Binary(Modulo, left, right).
4. La semántica acepta enteros y rechaza punteros o float si no está soportado.
5. El IR emite mod8 o mod16 según tipo.
6. El optimizador convierte $x \% 8$ en $x \& 7$ cuando es unsigned y potencia de dos.
7. El backend usa helper `__rt_mod_u8` si no hay caso inline.
8. El catálogo marca dependencia del helper.
9. Los tests cubren $7 \% 3$, $7 \% 1$, $x \% 8$, divisor cero constante y divisor dinámico.
10. La documentación describe política de división/módulo por cero.

Feature gates y perfiles

No todas las features deben estar siempre disponibles. Puede haber perfiles por target, por tamaño o por madurez. Por ejemplo, dynamic trig en precise puede estar deferred mientras compact y balanced están implementados.

```
$ picc --target pic16f628a --math-profile compact main.c  
$ picc --target pic16f877a --enable-float main.c
```

Regla de mantenimiento

Una feature incompleta debe fallar con un diagnóstico explícito, no generar código parcialmente incorrecto.

27. Estructura de repositorio y lectura con criterio

Un compilador no se entiende leyendo archivos al azar. Conviene seguir el flujo de datos: documentación, frontend, IR, backend, runtime, tests y reportes. El objetivo es conectar cada archivo con una fase del pipeline.

Orden recomendado

1. README: estado del proyecto, comandos y targets soportados.
2. DESIGN o documentación de arquitectura: decisiones globales y ABI.
3. docs/review.md o notas de fases: por qué se añadieron features.
4. src/frontend: lexer, parser y análisis semántico.
5. src/ir: tipos de IR, generación y optimizaciones.
6. src/backend/pic16: lowering, banking, paging y emisión.
7. runtime o helpers: rutinas incluidas bajo demanda.
8. tests: contratos reales del compilador.
9. reports: tamaño, memoria, cobertura de features y validación.

Qué buscar al leer código de compilador

Pregunta	Dónde suele responderse
¿Qué lenguaje soporta?	Parser, semantic tests, docs.
¿Qué tipos existen?	Type system, semantic analyzer, IR type definitions.
¿Cómo se pasan argumentos?	ABI docs, backend call lowering, tests.
¿Cómo se accede a SFR?	Target definitions, symbol table, backend memory lowering.
¿Qué helpers se emiten?	Runtime catalog, dependency resolver, memory report.
¿Cómo se valida?	Simulator tests, golden outputs, reports.

Dumps como documentación viva

Los dumps son una forma de leer el compilador ejecutándolo. Para una misma entrada, compara tokens, AST, IR, ASM, MAP y HEX. Cuando una feature no se entiende, crea un programa mínimo que la use y observa cómo cambia cada dump.

```
unsigned char x;

void main(void) {
    x = 3 + 4 * 5;
}
```

```
$ picc --dump-tokens expr.c  
$ picc --dump-ast expr.c  
$ picc --dump-ir expr.c  
$ picc --emit-asm expr.c  
$ picc --size expr.c
```

Consejo práctico

No leas el backend primero. Lee una feature de extremo a extremo. Empieza por un C mínimo, mira el token, el AST, el IR, el ASM y el test que lo valida.

28. Diseño de CLI, reports y experiencia de uso

La interfaz de línea de comandos es parte del compilador. Un CLI claro permite integrarlo en Makefiles, CI, scripts de flasheo y pruebas. Debe parecerse a herramientas conocidas cuando tenga sentido, pero sin prometer compatibilidad que no existe.

Opciones típicas

Opción	Uso
--target pic16f628a	Seleccionar dispositivo y sus límites.
-I include	Añadir ruta de includes.
-Wall -Wextra -Werror	Control de diagnósticos.
-O0/-O1/-O2	Nivel de optimización.
-o build/main.hex	Salida principal.
--dump-ast/--dump-ir	Depuración de fases.
--emit-asm	Listado ensamblador.
--memory-report	Resumen de ROM/RAM/stack/helpers.
--math-profile compact	Seleccionar perfil de math runtime.

Makefile mínimo

```
PIC := picc
TARGET := pic16f628a
CFLAGS := -Wall -Wextra -Werror -O2 -I include
SRC ?= blink.c
OUT ?= build/$(basename $(notdir $(SRC))).hex

all: $(OUT)

$(OUT): $(SRC)
    mkdir -p build
    $(PIC) --target $(TARGET) $(CFLAGS) -o $(OUT) $(SRC)

clean:
    rm -rf build
```

Reports de tamaño

Los reports deben ser suficientemente detallados para tomar decisiones. Si una actualización añade 500 palabras ROM, el usuario debe saber si vienen de trigonometría, división, prólogos de función o banking redundante.

```
Size report:
  user code: 418 words
  runtime helpers: 312 words
    __rt_f32_sincos_core: 215 words
    __rt_f32_cmp: 48 words
    __rt_mul_u8: 49 words
  vector/stubs: 18 words
  total: 748 words
```

Map y listado

El .map conecta símbolos con direcciones. El .lst conecta fuente, ensamblador y palabras codificadas. Ambos son fundamentales para depurar firmware real y para explicar qué hizo el compilador.

Experiencia de uso

Un compilador pequeño puede ser muy profesional si sus errores, reports y salidas son claras.

29. Taller guiado: traducir una función completa

Este capítulo recorre una función pequeña desde C hasta una forma cercana a PIC16. La intención no es fijar un backend exacto, sino ver la cadena lógica completa.

Código fuente

```
unsigned char limit(unsigned char value) {
    if (value > 10) {
        return 10;
    }

    return value + 1;
}
```

Tokens relevantes

```
KW_UNSIGNED KW_CHAR IDENT(limit) LPAREN
KW_UNSIGNED KW_CHAR IDENT(value) RPAREN LBRACE
KW_IF LPAREN IDENT(value) GREATER INT(10) RPAREN LBRACE
KW_RETURN INT(10) SEMICOLON RBRACE
KW_RETURN IDENT(value) PLUS INT(1) SEMICOLON
RBRACE EOF
```

AST simplificado

```
Function limit returns u8
  Param value: u8
  Body
    If
      Condition: Greater(Var(value), Int(10))
      Then: Return(Int(10))
      Return(Add(Var(value), Int(1)))
```

Semántica

- limit existe en scope global y devuelve unsigned char.
- value existe en scope de función y tiene tipo unsigned char.
- 10 cabe en unsigned char.
- value > 10 produce una condición booleana representada como entero/flag interno.
- Ambos return devuelven un valor compatible con unsigned char.

IR conceptual

```
function limit(value:u8) -> u8
entry:
```

```

    t0:u8 = load_arg value
    t1:bool = gt_u8 t0, 10
    branch_if_zero t1, after_if
then:
    return 10
after_if:
    t2:u8 = load_arg value
    t3:u8 = add8 t2, 1
    return t3

```

Lowering conceptual a PIC16

```

; compare value > 10
movf    value_slot, W
sublw   10          ; conceptual compare path may vary
; branch according to carry/zero policy
; if greater, return 10

return_ten:
    movlw 10
    return

after_if:
    movf    value_slot, W
    addlw   1
    return

```

El detalle exacto de comparación depende de cómo el backend modela flags. Lo importante es que cada fase reduce ambigüedad: el AST sabe que hay if; el IR sabe que hay branch; el backend sabe qué flags mirar; el codificador sabe los bits.

Tests recomendados

Entrada	Resultado esperado
0	1
9	10
10	11 si la condición es > 10
11	10
255	10

Aprendizaje

Un ejemplo pequeño permite ver todos los errores: precedencia, tipos, comparación unsigned, ramas, retorno y ABI.

30. Glosario ampliado

Este glosario recoge términos que en el documento original aparecían de forma breve y que conviene dominar para leer o diseñar el compilador.

Término	Definición didáctica
ABI	Contrato binario entre caller y callee: argumentos, retornos, preservación y frame.
AST	Árbol de sintaxis abstracta; estructura jerárquica del programa fuente.
Backend	Parte que convierte IR en operaciones del target y salida final.
Banking	Gestión de bancos de RAM o SFR en PIC16.
CFG	Control Flow Graph; grafo de bloques básicos y saltos.
Codificación	Conversión de una instrucción seleccionada en bits/palabras de máquina.
Constant folding	Evaluación de expresiones constantes durante compilación.
DCE	Dead Code Elimination; eliminación de código sin efecto observable.
Frontend	Parte que entiende el lenguaje fuente: lexer, parser y semántica.
Frame	Zona de memoria asociada a una llamada: argumentos, locales y temporales.
FSR/INDF	Mecanismo clásico de direccionamiento indirecto en PIC16.
Helper runtime	Rutina auxiliar emitida para operaciones no triviales.
Intel HEX	Formato textual para representar datos binarios programables.
IR	Representación intermedia entre AST y backend.
ISR	Rutina de servicio de interrupción.
Lexer	Fase que convierte caracteres en tokens.
Lowering	Conversión de operaciones abstractas a operaciones cercanas al target.
Paging	Gestión de páginas de memoria de programa para llamadas y saltos.
Parser	Fase que convierte tokens en estructura gramatical/AST.
Peephole	Optimización local sobre ventanas pequeñas de instrucciones.
SFR	Special Function Register; registro de control/periférico del microcontrolador.
Stack software	Pila implementada por el compilador en RAM, distinta de la pila hardware de retorno.
Token	Unidad léxica: identificador, literal, operador, palabra clave, etc.
Typed IR	IR donde cada temporal y operación tiene tipo explícito.
Vertical slice	Feature mínima que atraviesa todo el pipeline de extremo a extremo.

Mini chuleta de representaciones

Text:

```
x = a + 3;
```

Tokens:

```
IDENT(x), ASSIGN, IDENT(a), PLUS, INT(3), SEMICOLON
```

AST:

```
Assign(x, Add(Var(a), Int(3)))
```

Typed IR:

```
t0:u8 = load a  
t1:u8 = add8 t0, 3  
store x, t1
```

PIC16-style assembly:

```
movf a, W  
addlw 3  
movwf x
```

HEX:

```
Intel HEX records containing encoded instruction words
```

31. Conclusión: cómo pensar como quien diseña compiladores

pic16cc es pequeño comparado con GCC o LLVM, pero pedagógicamente es muy rico. Obliga a resolver problemas reales sin esconderse detrás de una arquitectura cómoda: tipos, memoria, llamadas, interrupciones, helpers, banking, paging, codificación, HEX, validación y tamaño.

La lección principal no es una lista de fases. La lección es una forma de pensar: reducir ambigüedad paso a paso, mantener contratos entre capas, medir costes, validar con programas reales y documentar límites. Un compilador no se vuelve sólido por soportar muchas features, sino por soportar pocas features con invariantes claras.

Principios finales

- Empieza por un vertical slice pequeño que llegue hasta HEX.
- No mezcles responsabilidades: lexer, parser, semántica, IR y backend deben tener fronteras claras.
- Define el ABI pronto; las llamadas aparecen en todas partes.
- Trata memoria, bancos y páginas como parte del lenguaje efectivo del target.
- Mide ROM, RAM, stack y helpers desde las primeras fases.
- Cada feature debe traer tests de éxito, error, tamaño y, si procede, simulación.
- Sé honesta con los límites: finite-only, rangos de trigonometría, precisión y features deferred son decisiones válidas si están documentadas.

Un último ejemplo mental

Cuando veas una línea como $y = \sin(x) + 1.0f$, no pienses solo en una suma. Piensa en tokens, AST, tipos float, llamada a helper, perfil de math, rango validado, core compartido, temporales de 32 bits, ABI, páginas, ROM, tolerancia y simulación. Esa es la diferencia entre escribir un traductor de texto y diseñar un compilador.

Cierre

Construir un compilador para PIC16 enseña precisamente porque obliga a hacer visible cada capa. Si entiendes esas capas, ya no estás copiando recetas: estás diseñando una herramienta.

Apéndice A. Plantilla de tests por fase

Esta plantilla sirve para añadir pruebas a medida que el compilador crece. La idea es que cada bug nuevo deje una prueba que lo impida volver.

Lexer

```
test "lex unsigned char declaration" {
    input = "unsigned char x;"
    expect_tokens = [KW_UNSIGNED, KW_CHAR, IDENT("x"), SEMICOLON, EOF]
}
```

Parser

```
test "multiplication binds tighter than addition" {
    input = "3 + 4 * 5"
    expect_ast = Add(Int(3), Mul(Int(4), Int(5)))
}
```

Semántica

```
test "reject undeclared identifier" {
    input = "void main(void) { x = 1; }"
    expect_error = "undeclared identifier x"
}
```

IR

```
test "emit add8 for unsigned char addition" {
    input = "unsigned char f(unsigned char a) { return a + 1; }"
    expect_ir_contains = "add8"
}
```

Backend y simulación

```
test "simulate arithmetic result" {
    compile "result = (3 * 4) + 5;"
    run_in_simulator()
    assert_ram("result") == 17
}
```

Tamaño

```
test "sinf emits one shared core" {
    compile "a = sinf(x); b = cosf(y);"
```

```
    assert_helper_count("__rt_f32_sincos_core") == 1  
}
```

Disciplina

Cada vez que añadas una feature, añade al menos un test por capa afectada. Si una capa no tiene test, esa capa acabará convirtiéndose en una caja negra.

Apéndice B. Errores frecuentes al construir pic16cc

1. Empezar por demasiada sintaxis

Soportar switch, structs, punteros y float antes de tener un recorrido mínimo estable suele generar deuda técnica. Primero el vertical slice; después el lenguaje.

2. Generar ASM desde el AST directamente

Funciona al principio, pero cada expresión compleja duplica lógica. Una IR sencilla evita que el backend tenga que entender C completo.

3. No definir ABI

Sin ABI, cada llamada y helper inventa su propia convención. Esto rompe en cuanto hay llamadas anidadas, varios argumentos o retorno de 16 bits.

4. Ignorar bancos hasta el final

Banking debe estar en el modelo desde el principio. Si lo añades tarde, tendrás que revisar todos los accesos a memoria.

5. Optimizar flags sin comprobar usos

En PIC16, muchas instrucciones modifican banderas. Sustituir una secuencia por otra aparentemente equivalente puede romper una rama posterior.

6. Emitir todos los helpers siempre

En microcontroladores, emitir runtime no usado es un desperdicio grave. El catálogo de helpers y la poda son parte de la arquitectura.

7. Validar solo texto

Que el ASM parezca correcto no significa que el HEX ejecute correctamente. Hay que simular o probar en hardware, especialmente para banking, paging e ISR.

8. No documentar límites

Un límite documentado es una decisión técnica. Un límite oculto es un bug esperando aparecer. Esto aplica a float finite-only, rango de trigonometría, ausencia de NaN/Inf, recursión o punteros a función.